

Fachliches Metamodell

1. Ziel des Metamodells

Dieses Metamodell beschreibt die **fachlichen Bausteine**, aus denen ein graphbasiertes Servicemodell aufgebaut werden soll. Es definiert:

- welche **Objekttypen** modelliert werden dürfen
- welche **Beziehungstypen** zwischen diesen Objekten zulässig sind
- welche **Eigenschaften** diese Objekte und Beziehungen besitzen
- welche **fachlichen Regeln** für die Modellierung gelten

Das Metamodell schafft damit eine einheitliche Grundlage für die Beschreibung komplexer Services, ihrer Teilservices, Abhängigkeiten, Varianten, Vorbedingungen und Provisionierungslogiken.

Es beschreibt **nicht** die konkrete technische Speicherung oder ein bestimmtes Werkzeug, sondern die fachliche Modellierungssprache.

2. Zweck des Metamodells

Das Metamodell verfolgt vier zentrale Ziele:

2.1 Einheitlichkeit

Alle Services sollen nach denselben Modellierungsprinzipien beschrieben werden.

2.2 Auswertbarkeit

Das Modell soll nicht nur lesbar, sondern auch fachlich auswertbar sein, z. B. zur Ableitung von Abhängigkeiten, Reihenfolgen und Blockaden.

2.3 Wiederverwendbarkeit

Servicebausteine sollen in mehreren Service-Kompositionen verwendet werden können.

2.4 Trennung der Sichten

Struktur, Abhängigkeiten, Regeln und Ausführung sollen sauber getrennt, aber miteinander verknüpfbar beschrieben werden.

3. Grundprinzipien des Metamodells

3.1 Hierarchie und Graph sind zwei verschiedene Dinge

Die Zugehörigkeit eines Bausteins zu einem Service ist nicht dasselbe wie seine Abhängigkeit von einem anderen Baustein.

3.2 Knoten und Beziehungen sind gleichwertige Modellbestandteile

Nicht nur die Elemente selbst, sondern auch deren Beziehungen müssen fachlich klar typisiert und beschrieben werden.

3.3 Fachliche Modellierung vor technischer Modellierung

Das Metamodell beschreibt zuerst die fachliche Struktur. Die technische Umsetzung folgt später.

3.4 Versionierbarkeit

Jedes Modell und jeder modellierte Baustein muss versionierbar sein.

3.5 Erweiterbarkeit

Das Metamodell soll so gestaltet sein, dass weitere Knoten- oder Beziehungstypen später ergänzt werden können.

4. Überblick über die Metamodell-Elemente

Das Metamodell besteht aus vier Ebenen:

1. **Modellkontext**
 2. **Knotentypen**
 3. **Beziehungstypen**
 4. **Regeln und Attribute**
-

5. Modellkontext

5.1 Modell

Das Modell ist die Gesamtheit aller Knoten, Beziehungen und Regeln, die eine fachliche Service-Komposition beschreiben.

Attribute des Modells

Attribut	Beschreibung
Modell-ID	Eindeutige Identifikation des Modells
Name	Name des Modells
Beschreibung	Fachliche Beschreibung
Version	Versionsstand des Modells
Status	z. B. Entwurf, freigegeben, ausser Betrieb
Gültig ab	Beginn der Gültigkeit
Gültig bis	Ende der Gültigkeit
Verantwortliche Rolle	Fachliche Verantwortung
Bezugsservice	Welcher Service durch dieses Modell beschrieben wird

6. Knotentypen

6.1 Überblick

Ein Knoten repräsentiert einen fachlich relevanten Baustein innerhalb eines Servicegraphen.

Das Metamodell unterscheidet folgende Knotentypen:

1. Service-Knoten
 2. Sub-Service-Knoten
 3. Aktivitäts-Knoten
 4. Vorbedingungs-Knoten
 5. Entscheidungs-Knoten
 6. Validierungs-Knoten
 7. Manueller Knoten
 8. Kompensations-Knoten
 9. Zustands-Knoten
 10. Referenz-Knoten
-

6.2 Gemeinsame Basisattribute aller Knoten

Jeder Knoten besitzt mindestens die folgenden allgemeinen Attribute:

Attribut	Beschreibung
Knoten-ID	Eindeutige Identifikation
Name	Fachlicher Name
Beschreibung	Fachliche Erläuterung
Knotentyp	Typ gemäss Metamodell
Status	Modellierungsstatus des Knotens
Pflicht/Optional	Ist der Knoten immer erforderlich oder nur optional
Aktivierungsregel	Unter welchen Bedingungen wird der Knoten aktiv
Variante	Für welche Varianten gilt der Knoten
Verantwortliche Rolle	Wer fachlich für den Knoten zuständig ist
Wiederverwendbar	Ja/Nein
Version	Versionsstand
Tags/Kategorien	Zur fachlichen Gruppierung

6.3 Service-Knoten

Zweck

Repräsentiert einen aus fachlicher Sicht sichtbaren oder bestellbaren Service.

Typische Eigenschaften

- bildet den fachlichen Einstiegspunkt
- kann andere Knoten enthalten
- kann Varianten besitzen
- kann mehrere Abhängigkeitsbeziehungen zu anderen Knoten haben

Zusätzliche Attribute

Attribut	Beschreibung
Service-Code	Fachlicher Identifikator
Serviceziel	Beschreibt den angestrebten Nutzen
Zielgruppe	Für wen der Service bestimmt ist

6.4 Sub-Service-Knoten

Zweck

Repräsentiert einen fachlichen Bestandteil eines Service.

Typische Eigenschaften

- gehört fachlich zu einem Service oder Sub-Service
- kann selbst wieder aus weiteren Knoten bestehen
- ist wiederverwendbar

Zusätzliche Attribute

Attribut	Beschreibung
Parent-Service-ID	Fachliche Zugehörigkeit
Fachlicher Zweck	Beitrag zum Gesamtservice

6.5 Aktivitäts-Knoten

Zweck

Repräsentiert einen ausführbaren oder fachlich auszulösenden Schritt.

Typische Eigenschaften

- wird im Rahmen der Provisionierung ausgeführt
- kann automatisch oder manuell sein
- kann Vorbedingungen, Validierungen und Kompensationen besitzen

Zusätzliche Attribute

Attribut	Beschreibung
Ausführungsart	automatisch / manuell / hybrid
Ergebnisart	erwartetes fachliches Ergebnis
Idempotenz erwartet	Ja/Nein
Retry-fähig	Ja/Nein

6.6 Vorbedingungs-Knoten

Zweck

Repräsentiert eine Bedingung, die erfüllt sein muss, bevor ein anderer Knoten aktiviert oder ausgeführt werden darf.

Typische Eigenschaften

- kann fachlich oder technisch sein
- kann mehrere nachgelagerte Knoten beeinflussen

Zusätzliche Attribute

Attribut	Beschreibung
Bedingungstyp	fachlich / technisch / organisatorisch / zeitlich
Prüfart	manuell / automatisch
Muss-Kriterium	Ja/Nein

6.7 Entscheidungs-Knoten

Zweck

Repräsentiert eine Regel oder Auswahl, die über den weiteren Verlauf entscheidet.

Typische Eigenschaften

- steuert Varianten
- aktiviert alternative Pfade
- bewertet Parameter oder Kontextinformationen

Zusätzliche Attribute

Attribut	Beschreibung
Entscheidungslogik	Fachliche Beschreibung der Entscheidung
Entscheidungsbasis	Welche Informationen ausgewertet werden
Ergebniswerte	Mögliche Resultate

6.8 Validierungs-Knoten

Zweck

Prüft, ob ein gewünschter fachlicher oder technischer Zustand erreicht wurde.

Typische Eigenschaften

- steht meist nach einer Aktivität
- kann blockierend oder nicht blockierend sein
- liefert Ergebnis erfolgreich / fehlgeschlagen / unklar

Zusätzliche Attribute

Attribut	Beschreibung
Validierungsziel	Was geprüft wird
Validierungsart	automatisch / manuell
Blockierend	Ja/Nein

6.9 Manueller Knoten

Zweck

Repräsentiert eine Tätigkeit, die durch eine Person oder Rolle durchgeführt werden muss.

Typische Eigenschaften

- dient Freigaben, Klärungen, Nachbearbeitung oder Ausnahmebehandlung
- kann den weiteren Graphen blockieren oder freigeben

Zusätzliche Attribute

Attribut	Beschreibung
Bearbeitungsrolle	Zuständige Rolle
Frist relevant	Ja/Nein
Eskalationsregel	Beschreibung der Eskalation

6.10 Kompensations-Knoten

Zweck

Beschreibt eine fachliche Gegenmassnahme, falls ein anderer Knoten fehlschlägt oder ein Rückbau nötig wird.

Typische Eigenschaften

- ist mit einem oder mehreren Knoten fachlich verknüpft
- bildet nicht zwingend eine exakte technische Rückabwicklung ab
- kann selbst weitere Bedingungen haben

Zusätzliche Attribute

Attribut	Beschreibung
Kompensationsziel	Welcher Zustand korrigiert werden soll
Kompensationsart	rückgängig machen / deaktivieren / kennzeichnen / manuell bereinigen
Pflicht bei Fehler	Ja/Nein

6.11 Zustands-Knoten

Zweck

Repräsentiert einen fachlich relevanten Zustand, der im Modell explizit referenziert werden soll.

Typische Eigenschaften

- eignet sich z. B. für „Konto vorhanden“, „Mailbox aktiv“, „Genehmigung erteilt“
- kann als Referenzpunkt für Aktivierung oder Validierung dienen

Zusätzliche Attribute

Attribut	Beschreibung
Zustandstyp	fachlich / technisch / organisatorisch
Zielzustand	Beschreibung des gewünschten Zustands

6.12 Referenz-Knoten

Zweck

Repräsentiert einen wiederverwendbaren oder extern referenzierten Modellbaustein.

Typische Eigenschaften

- dient zur Wiederverwendung
- verweist auf einen bereits definierten Knoten oder Baustein
- reduziert Duplikate

Zusätzliche Attribute

Attribut	Beschreibung
Referenzziel	ID des referenzierten Bausteins
Referenzart	intern / extern

Bindungsart	statisch / versioniert
-------------	------------------------

7. Beziehungstypen

7.1 Überblick

Beziehungen verbinden Knoten miteinander und machen deren fachliche Zusammenhänge explizit.

Jede Beziehung besitzt:

- eine Richtung
- einen fachlichen Beziehungstyp
- optional Bedingungen
- eine Stärke oder Verbindlichkeit

7.2 Gemeinsame Basisattribute aller Beziehungen

Attribut	Beschreibung
Beziehungs-ID	Eindeutige Identifikation
Quell-Knoten-ID	Ursprung
Ziel-Knoten-ID	Ziel
Beziehungstyp	Typ gemäss Metamodell
Verbindlichkeit	hart / weich
Bedingung	Optionaler Aktivierungskontext
Beschreibung	Fachliche Erläuterung
Version	Versionsstand

7.3 Beziehungstyp „besteht aus“

Bedeutung

Der Quellknoten enthält oder gliedert den Zielknoten fachlich ein.

Beispiel

Service besteht aus Sub-Service.

Eigenschaften

- strukturelle Beziehung
- nicht automatisch identisch mit Ausführungsreihenfolge

7.4 Beziehungstyp „depends on“

Bedeutung

Der Zielknoten darf erst aktiv oder ausführbar werden, wenn der Quellknoten erfolgreich erfüllt oder abgeschlossen ist.

Beispiel

Mailbox bereitstellen depends on Konto bereitstellen.

Eigenschaften

- zentrale Abhängigkeitsbeziehung
 - relevant für Reihenfolgeermittlung
 - bei harter Ausprägung azyklisch zu modellieren
-

7.5 Beziehungstyp „enables“

Bedeutung

Der Quellknoten schaltet den Zielknoten fachlich frei.

Beispiel

Genehmigung erteilt enables Bereitstellung starten.

Eigenschaften

- eher aktivierend als strikt logisch voraussetzend
 - kann zusätzlich zu depends on verwendet werden
-

7.6 Beziehungstyp „blocks“

Bedeutung

Der Quellknoten oder dessen Zustand verhindert die Aktivierung oder Ausführung des Zielknotens.

Beispiel

Fehlende Stammdaten blocks Konto anlegen.

Eigenschaften

- eignet sich für Ausnahme- oder Negativlogik
 - sollte sparsam und bewusst eingesetzt werden
-

7.7 Beziehungstyp „validates“

Bedeutung

Der Quellknoten prüft den Zielknoten oder dessen Ergebnis.

Beispiel

Mailbox validieren validates Mailbox bereitstellen.

7.8 Beziehungstyp „compensates“

Bedeutung

Der Quellknoten beschreibt die fachliche Kompensation des Zielknotens.

Beispiel

Mailbox deaktivieren kompensates Mailbox bereitstellen.

7.9 Beziehungstyp „alternative to“

Bedeutung

Zwei Knoten stehen in einer Alternativbeziehung, von denen fachlich nur einer oder eine definierte Teilmenge aktiv sein darf.

Beispiel

U-Account alternative to X-Account.

7.10 Beziehungstyp „requires condition“

Bedeutung

Der Zielknoten wird nur aktiv, wenn eine zusätzliche Bedingung erfüllt ist.

Beispiel

Zusatzschritt requires condition „externer Mitarbeiter“.

7.11 Beziehungstyp „produces state“

Bedeutung

Ein Knoten erzeugt einen Zustand, der später weiterverwendet wird.

Beispiel

Konto anlegen produces state „Konto vorhanden“.

7.12 Beziehungstyp „consumes state“

Bedeutung

Ein Knoten benötigt einen zuvor vorhandenen Zustand.

Beispiel

Mailbox anlegen consumes state „Konto vorhanden“.

8. Regeln als eigenes Metamodell-Element

Neben Knoten und Beziehungen braucht das Metamodell fachliche Regeln.

8.1 Regel

Eine Regel ist eine fachliche Aussage, die die Aktivierung, Gültigkeit, Auswahl oder Bewertung von Modellteilen steuert.

Basisattribute einer Regel

Attribut	Beschreibung
Regel-ID	Eindeutige ID
Name	Fachlicher Name
Beschreibung	Erläuterung
Regeltyp	Aktivierung / Validierung / Variantenregel / Konsistenzregel / Kompensationsregel
Ausdruck	Fachlicher oder strukturierter Regelausdruck
Geltungsbereich	Für welche Knoten/Beziehungen die Regel gilt
Verbindlichkeit	Muss / Soll / Kann

8.2 Regeltypen

Aktivierungsregel

Steuert, wann ein Knoten oder eine Beziehung aktiv ist.

Variantenregel

Legt fest, welche Knoten für welche Variante gelten.

Validierungsregel

Definiert, wann ein Ergebnis als erfolgreich oder ausreichend gilt.

Konsistenzregel

Prüft, ob das Modell fachlich gültig ist.

Kompensationsregel

Definiert, wann und wie Kompensationen anzuwenden sind.

9. Fachliche Sicht auf Varianten im Metamodell

Varianten sollen nicht durch vollständige Duplikation ganzer Modelle beschrieben werden, sondern durch gezielte Unterschiede.

9.1 Variante

Eine Variante beschreibt eine bestimmte Ausprägung eines Service.

Attribute der Variante

Attribut	Beschreibung
Varianten-ID	Eindeutige ID
Name	Variantenname

Beschreibung	Fachliche Erläuterung
Kontext	Wann die Variante gilt
Regelbezug	Welche Regeln die Variante steuern

9.2 Variantenverwendung

Varianten können auf drei Ebenen wirken:

- auf Knoten
- auf Beziehungen
- auf Regeln

Damit kann z. B. modelliert werden, dass ein bestimmter Aktivitätsknoten nur für U-Accounts gilt oder dass bestimmte Beziehungen nur für externe Mitarbeitende aktiv sind.

10. Fachliche Konsistenzregeln des Metamodells

Das Metamodell braucht verbindliche Modellierungsregeln.

10.1 Eindeutigkeit

Jeder Knoten, jede Beziehung und jede Regel muss eindeutig identifizierbar sein.

10.2 Typisierung

Jeder Knoten und jede Beziehung muss genau einen definierten Typ besitzen.

10.3 Keine impliziten Abhängigkeiten

Abhängigkeiten müssen explizit modelliert werden und dürfen nicht nur aus Freitext hervorgehen.

10.4 Zyklusfreiheit bei harten Abhängigkeiten

Beziehungen vom Typ depends on mit Verbindlichkeit „hart“ dürfen keinen Zyklus bilden.

10.5 Referenzierbarkeit von Regeln

Wenn eine Aktivierung oder Variante durch eine Regel gesteuert wird, muss diese Regel explizit referenziert sein.

10.6 Trennung von Struktur und Ausführung

Eine besteht-aus-Beziehung allein darf nicht als Ausführungsreihenfolge interpretiert werden.

10.7 Nachvollziehbare Verantwortlichkeit

Für modellrelevante Knoten muss eine verantwortliche Rolle angegeben sein.

11. Empfohlenes Minimalset für die erste Einführung

Für einen pragmatischen Start würde ich nicht alle Elemente sofort verpflichtend machen, sondern mit einem Minimalset beginnen.

11.1 Minimale Knotentypen

- Service-Knoten
- Sub-Service-Knoten
- Aktivitäts-Knoten
- Vorbedingungs-Knoten

- Validierungs-Knoten
- Manueller Knoten

11.2 Minimale Beziehungstypen

- besteht aus
- depends on
- validates
- requires condition

11.3 Minimale Regeltypen

- Aktivierungsregel
- Variantenregel
- Konsistenzregel

Das reicht meist schon aus, um erste Servicegraphen belastbar zu modellieren.

12. Beispielhafte Metamodell-Darstellung

12.1 Vereinfachte Struktur

```
Modell
enthält Knoten
  Service-Knoten
  Sub-Service-Knoten
  Aktivitäts-Knoten
  Vorbedingungs-Knoten
  Entscheidungs-Knoten
  Validierungs-Knoten
  Manueller Knoten
  Kompensations-Knoten
  Zustands-Knoten
  Referenz-Knoten

enthält Beziehungen
besteht aus
depends on
enables
blocks
validates
compensates
alternative to
requires condition
produces state
consumes state

enthält Regeln
Aktivierungsregel
Variantenregel
Validierungsregel
Konsistenzregel
Kompensationsregel
```

13. Nutzen des Metamodells

Dieses Metamodell ermöglicht:

- eine einheitliche Sprache für komplexe Service-Kompositionen
- die strukturierte Modellierung von Hierarchie und Abhängigkeit
- die Wiederverwendung von Servicebausteinen

- die automatisierte Ableitung von ausführbaren Schritten
 - die Prüfung fachlicher Konsistenz
 - die Grundlage für spätere technische Umsetzung in Datenmodellen, Graphspeichern oder Orchestrierungslösungen
-

14. Zusammenfassung

Das fachliche Metamodell definiert die Modellierungssprache für graphbasierte Service-Kompositionen.

Im Zentrum stehen:

- **Knoten** als fachliche Bausteine
- **Beziehungen** als explizite Verknüpfungen
- **Regeln** zur Steuerung von Aktivierung, Varianten, Validierung und Konsistenz

Dadurch wird eine Modellierung möglich, die sowohl **verständlich** als auch **auswertbar** ist und die deutlich über eine rein hierarchische Zerlegung hinausgeht.